01 SHARED SETUP

Start the backend

C2 Device Manager and C3 Data Management | Rishi Sheth | CMPE 281 Group 8. Commands use Bash on Linux, macOS or WSL. Use Python 3.12, Node 18+ (a supported LTS version preferred), and Docker Desktop or Docker Engine with Compose.

> Use feature/rishi-c2-c3. This guide matches commit 7d74eaa, checked November 9, 2026. C1 is still a separate branch: this branch uses the role dropdown, not JWT login. Stop older app processes before changing branches.

1. Get the component branch

```
git fetch origin
git switch feature/rishi-c2-c3
git pull --ff-only origin feature/rishi-c2-c3
```

2. Start the databases - from the repo root

```
docker compose -f infra/docker-compose.yml up -d
docker compose -f infra/docker-compose.yml ps
```

Wait until PostgreSQL and MongoDB are healthy. Docker is needed for this database recipe, but not for the automated tests.

3. First-time backend setup - Terminal 1

```
cd backend
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
test -f .env || cp .env.example .env
```

Already have the backend environment? Activate it and update requirements. Keep your existing .env. On Windows PowerShell, activate with .\.venv\Scripts\Activate.ps1; the remaining examples here use Bash.

4. Set the Wi-Fi encryption key once

```
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
```

Paste the generated value into WIFI_ENCRYPTION_KEY in backend/.env. Keep it private and reuse it after restarts. It is needed for password-protected Wi-Fi. Leave DEVICE_SWEEPER_ENABLED=true and PROCESSING_WORKER_ENABLED=true.

5. Migrate, seed and run - leave Terminal 1 open

```
alembic upgrade head
python -m app.db.seed
uvicorn app.main:app --reload
```

API docs: http://localhost:8000/docs. The migration head is 0002_device_commands. The audio worker and offline sweeper start with the API; do not start a second worker for this setup. Use one API process.

02 DEVICES AND COMMANDS

Open the UI and check C2

6. Start the frontend - Terminal 2, from the repo root

```
cd frontend
npm ci
test -f .env.local || cp .env.example .env.local
npm run dev -- --port 5173 --strictPort
```

Confirm frontend/.env.local has VITE_USE_MOCK=false and VITE_API_BASE_URL=http://localhost:8000. Restart Vite after changing either value. Open http://localhost:5173/devices.

Choose the right role

Select IoT Staff in the header to add/edit/delete devices, change Wi-Fi and queue commands. Cloud Staff can read devices and retry failed data records. The C1 login accounts are not used on this branch.

Pages: /devices (inventory), /devices?tab=data (processing monitor), /simulator?car_id=1 (camera, position and commands). Owners are locked out of these staff pages; the API separately supports owner-scoped device reads.

7. Confirm device access - Terminal 3

```
curl -sS 'http://localhost:8000/devices?q=mic-1' \
  -H 'X-Role: iot_staff' -H 'X-User-Id: 3'
```

A fresh seed creates car 1, microphone mic-1 (device 1), and camera cam-1 (device 5). Existing databases can have different IDs: use the returned car_id/id. Registering a new device in the UI shows its random device key once; save it before closing.

8. Send a heartbeat using the fresh-seed demo device

```
CAR_ID=1; DEVICE_ID=1
DEVICE_KEY=dev-device-key-mic-1
curl -sS -X POST "http://localhost:8000/devices/$DEVICE_ID/heartbeat" \
  -H "X-Device-Key: $DEVICE_KEY" -H 'Content-Type: application/json' \
  -d '{"firmware":"setup-check","lat":37.3352,"lon":-121.8811}'
```

For a newly registered device, replace CAR_ID, DEVICE_ID and DEVICE_KEY with its returned values. Expect HTTP 200 with server_time and commands. Demo keys are for local seeded devices only. A seed status of online is not a live heartbeat.

9. Queue and acknowledge a command

In Device Manager, open the device and send Start capture. Send another heartbeat; its reply contains the command_id. Replace COMMAND_ID below with that number. For a manual smoke test, acknowledge receipt; a real edge client must apply the action before acknowledging success.

```
COMMAND_ID=1  # replace with the command_id from the heartbeat
curl -sS -X POST \
  "http://localhost:8000/devices/$DEVICE_ID/commands/$COMMAND_ID/ack" \
  -H "X-Device-Key: $DEVICE_KEY" -H 'Content-Type: application/json' \
  -d '{"ok":true}'
```

Command history should show acknowledged. Queueing the same command twice while it is queued returns 409; acknowledging an already closed command also returns 409. Commands expire after ten minutes.

03 DATA MANAGEMENT

Enable real audio uploads

> S3 is required for real uploads, inference and camera storage. DEV_FAKE_BACKUP and LOCAL_AUDIO_DIR do not fake C3 ingestion. You can inspect the UI and test C2 inventory/heartbeats without S3.

1. Configure S3 on the backend machine

Stop the API in Terminal 1 (Ctrl+C). Use the team's AWS profile, exported credentials or an EC2 instance role. For a named profile, run export AWS_PROFILE=autosound-dev in that terminal if the profile exists. Set the actual bucket/region below, then restart uvicorn in Terminal 1. Exports in Terminal 3 do not update the running API; undeclared access-key entries in backend/.env are not exported to boto3.

```
# In backend/.env, use your actual bucket and its region:
S3_BUCKET=your-team-audio-bucket
AWS_REGION=us-west-2
```

The runtime needs s3:PutObject and s3:GetObject on the bucket's audio/* and frames/* objects. Keep the bucket private; playback uses presigned links. S3 bucket health/stat checks also need s3:ListBucket. Never share AWS keys in the setup files.

> For playback of newly uploaded alerts, remove/comment LOCAL_AUDIO_DIR=.local_audio in backend/.env and unset any exported LOCAL_AUDIO_DIR, then restart the API. Do not leave LOCAL_AUDIO_DIR= empty: an empty path does not disable local-audio mode. Raw C3 record audio already uses S3.

2. Check the active model - from backend/

```
python - <<'PY'
from app.services.orchestrator import get_orchestrator
print(get_orchestrator().reload_model().version_label)
PY
```

Expect cnn-v1 after seeding. Its weights/config/labels are in the repo; normal setup does not require model training or export. Run the API again after this check if it was stopped.

3. Upload one clip - Terminal 3, backend environment active

Use CAR_ID, DEVICE_ID and DEVICE_KEY from page 2. Supply a valid non-empty WAV or MP3, at most 10 MiB and 15 seconds. Replace the sample file path below; for MP3 use type=audio/mpeg.

```
CLIP_ID=$(python -c 'import uuid; print(uuid.uuid4())')
curl -sS -X POST http://localhost:8000/device-data \
  -H "X-Device-Key: $DEVICE_KEY" \
  -F "audio_file=@/path/to/clip.wav;type=audio/wav" \
  -F "car_id=$CAR_ID" -F "device_id=$DEVICE_ID" \
  -F "clip_id=$CLIP_ID"
```

Expect HTTP 202 and a record_id. Repeat the upload with the same CLIP_ID to get HTTP 200 and the original record ID. Keep that UUID when retrying an uncertain network result; generate a new UUID only for a new clip.

4. Watch the processing monitor

Open /devices?tab=data. A new record moves pending -> processing -> processed; its detail links to the resulting alert for Cloud Staff. Failures retry up to three attempts, then stay failed. Only Cloud Staff can use Retry. The seed creates alerts, not data records, so an initially empty Data tab is expected.

The worker polls every two seconds and uses a 120-second processing lease. Monitor the backend terminal for startup/model problems; no separate demo_data.py process is needed for C2/C3.

04 EDGE CLIENT

Optional CARLA demo

CARLA is not needed for the web UI or unit tests. For this demo you need a running, advancing CARLA server, an existing vehicle actor, S3 from page 3, registered microphone/camera devices on the same car, and five short recordings.

1. Match the client to the simulator

```
# Example only: use this when your server is CARLA 0.9.16.
python -m pip install carla==0.9.16
```

Use Python 3.12 for the CARLA 0.9.16 client and matching client/server versions. Its published wheels cover Windows and Linux x86-64, not macOS. A Mac can run the web app; run the edge client on a compatible machine. Pillow is already in backend requirements.

2. Prepare the five scenario clips - from backend/

```
mkdir -p .local_edge/clips
cp ../docs/setup/c2-c3/scenarios.example.json .local_edge/scenarios.json
```

Put emergency.wav, in_vehicle.wav, human.wav, traffic.wav and environmental.wav in .local_edge/clips/, or edit the copied JSON to point at your own WAV/MP3 paths. Paths are relative to the JSON file. Each clip must meet the page 3 limits.

3. Start the edge client - Terminal 3

These are fresh-seed local demo IDs/keys. Replace them if you registered different devices. Replace actor 42 with an existing CARLA vehicle ID; the adapter does not create a vehicle or start the server.

```
export EDGE_API_URL=http://localhost:8000
export EDGE_CAR_ID=1
export EDGE_DEVICE_ID=1
export EDGE_DEVICE_KEY=dev-device-key-mic-1
export EDGE_CAMERA_DEVICE_ID=5
export EDGE_CAMERA_DEVICE_KEY=dev-device-key-cam-1
python -m edge.carla_client \
  --actor-id 42 --camera \
  --scenarios .local_edge/scenarios.json
```

Omit --camera and its two EDGE_CAMERA variables for audio/position only. For a remote CARLA server, add --host with its address. CARLA RPC uses 2000; Traffic Manager uses 8001 to avoid the API on 8000.

If the edge client runs on another machine, set EDGE_API_URL to the backend's reachable address. Start that backend with uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload and permit port 8000 only on a trusted network. The current role-header auth is for development, not public exposure.

4. Try the Simulator page as IoT Staff

Open /simulator?car_id=1, select the microphone, then Start route, Stop route or Simulate event. Scenario commands forward the matching recording through C3; they do not synthesize sound or record a physical microphone. A heartbeat within 90 seconds is required before these commands can be queued.

> The buffer retains up to 200 clips and uploads one per heartbeat pass during recovery. Camera capture can be stopped independently. Remote reboot is unsupported. Wi-Fi settings are saved encrypted, but actual OS Wi-Fi application needs an explicit apply hook; the default client reports failure rather than claiming it applied them.

05 QUICK REFERENCE

Tests and troubleshooting

Tests - no Docker, AWS or CARLA needed

```
cd backend
source .venv/bin/activate
python -m pytest
alembic heads
```

Verified November 9, 2026 on feature/rishi-c2-c3 at 7d74eaa: 394 passed, 10 skipped; one Alembic head, 0002_device_commands. The skipped tests need a throwaway PostgreSQL database. Do not point TEST_DATABASE_URL at real data: those tests drop/recreate the schema. C1's branch has a different test count.

Common problems

Symptom	What to check
/devices returns 404	Use feature/rishi-c2-c3 and restart the API from backend/. Main does not yet contain these endpoints.
401 or 403	Staff API calls use X-Role/X-User-Id; edge calls use X-Device-Key. IoT edits/commands; Cloud reads/retries. C1 JWT login is a separate branch.
Device data is empty	Expected after seeding. Upload a clip to create a C3 record.
Wi-Fi save returns 503	Set a valid WIFI_ENCRYPTION_KEY and restart the API. Do not regenerate it on each start.
Upload returns 503	Check the real bucket/region, backend AWS credentials and PutObject permission. Local-audio/fake-backup flags do not fake ingestion.
New alert audio returns 404	Remove/comment LOCAL_AUDIO_DIR=.local_audio and restart the API so new alerts use S3 playback.
Record ends failed	Check S3 GetObject access and the cnn-v1 model check on page 3. Then retry as Cloud Staff.
Simulator command returns 409	Send a recent heartbeat or start the edge client. A seeded online status is not enough; duplicate queued commands also return 409.
No camera frame	Use a registered camera key, --camera, a live existing CARLA actor and working S3 access.
Port 5432 is busy	Change only the local compose host mapping to 5434:5432 and DATABASE_URL's host port to 5434. Recreate the service and restart the API; keep the container port 5432.
Frontend cannot connect	Keep VITE_USE_MOCK=false, API URL localhost:8000 and browser URL localhost:5173. Restart Vite after env edits; stop the old process if 5173 is occupied.

For deeper store health, call GET /admin/db/health as Cloud Staff and inspect its JSON. /health checks PostgreSQL only. Keep local port/credential edits out of commits, and do not delete Docker volumes to resolve a port conflict.

Scope and references

Live CARLA, AWS and real PostgreSQL still need a deployment smoke test; automated checks use isolated stores/transports. This guide is for the current C2/C3 branch, not a merged C1+C2+C3 release. Repo references: backend/README.md, backend/edge/README.md, backend/.env.example and frontend/.env.example.

CARLA 0.9.16 package and supported wheels

AWS S3 operation permissions